A Practical Guide to AXI4-Stream and AXI4-Stream Video

Interfaces, protocol rules and RTL design patterns

Yosel Balibrea Lastre

August 28, 2026

Revision 0.1 — draft

Contents

About this guide vii

I AXI4-Stream Fundamentals 1

1 Introduction to Streaming Interfaces 2
1.1 Why a streaming interface? . . . 2
1.2 Where AXI4-Stream sits in the AMBA family . . . 2
1.3 Terminology: transfer, packet, frame, stream . . . 2
1.4 Roadmap of the document . . . 2

2 The AXI4-Stream Protocol 3
2.1 Signal set . . . 3
2.1.1 Required vs. optional signals . . . 3
2.1.2 Byte qualifiers: data, position and null bytes . . . 3
2.2 The VALID/READY handshake . . . 3
2.2.1 Case 1: master first . . . 3
2.2.2 Case 2: slave first . . . 4
2.2.3 Case 3: simultaneous assertion . . . 4
2.3 Packet boundaries with `TLAST` . . . 4
2.4 Routing with `TID` and `TDEST` . . . 4
2.5 Sideband information with `TUSER` . . . 4
2.6 Reset behaviour . . . 4
2.7 Interface compliance checklist . . . 4

3 Timing, Throughput and Backpressure 5
3.1 Throughput model . . . 5
3.2 Bubbles and stalls . . . 5
3.3 Registering the interface . . . 5
3.3.1 Naive output register (breaks throughput) . . . 5
3.3.2 The skid buffer / register slice . . . 5
3.4 FIFOs and elastic buffering . . . 6
3.5 Deadlock and starvation . . . 6

4 Writing AXI4-Stream Masters and Slaves 7
4.1 A minimal master . . . 7
4.2 A minimal slave . . . 7
4.3 Pass-through and processing elements . . . 7
4.3.1 Zero-latency (combinational) processing . . . 7
4.3.2 Pipelined processing with backpressure . . . 7
4.4 Rate-changing elements . . . 7

4.5 Parameterization and reuse conventions 7

5 Stream Infrastructure Blocks 8
5.1 Register slice 8
5.2 Data FIFO 8
5.3 Width converter 8
5.4 Broadcaster (1-to-N) 8
5.5 Combiner and switch (N-to-1, N-to-M) 8
5.6 Subset converter 8

II AXI4-Stream Video 9

6 Digital Video Fundamentals 10
6.1 Raster scan, blanking and active video 10
6.2 Video timing parameters 10
6.3 Colour spaces and pixel formats 10
6.4 Bandwidth budgeting 10

7 The AXI4-Stream Video Protocol 11
7.1 Mapping video onto AXI4-Stream 11
7.2 Start of frame and end of line 11
7.3 Pixel packing in `TDATA` 11
7.3.1 One pixel per clock 11
7.3.2 Multiple pixels per clock 11
7.3.3 Component ordering and padding 11
7.4 Frame synchronisation and error recovery 12
7.5 Differences from generic AXI4-Stream 12

8 Video Timing and the Boundary Blocks 13
8.1 Video Timing Controller (VTC) 13
8.2 From parallel video to AXI4-Stream 13
8.3 From AXI4-Stream to parallel video 13
8.4 Clock domains in a video pipeline 13

9 Building a Video Processing Pipeline 14
9.1 Anatomy of a pipeline 14
9.2 Point operations 14
9.3 Line-based operations 14
9.4 Neighbourhood operations 14
9.5 Frame buffers and DMA 14
9.6 Worked example: grayscale converter 14

10 Verification and Debug 15
10.1 Testbench architecture 15
10.2 Randomised backpressure and valid throttling 15
10.3 Protocol checkers and assertions 15
10.4 Image-level checking 15
10.5 On-hardware debug 15
10.6 Symptom-to-cause table 15

A Signal Quick Reference 16

B Design Review Checklist **17**

C Glossary **18**

List of Figures

2.1 Basic AXI4-Stream handshake with back-to-back transfers. 4

7.1 Placement of SOF and EOL over a small frame. 11

9.1 Skeleton of an AXI4-Stream video pipeline. 14

List of Tables

1.1 Comparison of the AMBA AXI interface flavours. 2

2.1 AXI4-Stream signal summary. 3

6.1 Common video timing modes. 10

7.1 Video-specific interpretation of the AXI4-Stream signals. 11

10.1 Debugging AXI4-Stream video pipelines. 15

About this guide

Scope

[**TODO**] What this document covers: the AXI4-Stream protocol, its video profile, and how to write/verify RTL that speaks both.

Audience and prerequisites

[**TODO**] Assumed background: synchronous digital design, Verilog/SystemVerilog, basic FPGA toolchain use. No prior AXI knowledge assumed.

Conventions used

- Signal names are written as `TVALID`, `TREADY`, ...
- Master-side ports are prefixed `m_axis_`, slave-side `s_axis_`.
- All code examples are synthesizable Verilog-2001 unless stated otherwise.
- [**TODO**] Add: active-low reset convention, clock naming.

Accompanying source code

[**TODO**] Point at the repository layout, e.g. `m_axis.v`, `s_axis.v`, testbenches and simulation scripts.

Part I

AXI4-Stream Fundamentals

Chapter 1

Introduction to Streaming Interfaces

1.1 Why a streaming interface?

[**TODO**] Memory-mapped vs. streaming data movement; when each is appropriate.

1.2 Where AXI4-Stream sits in the AMBA family

[**TODO**] AXI4 / AXI4-Lite / AXI4-Stream comparison table.

Table 1.1: Comparison of the AMBA AXI interface flavours.

	AXI4	**AXI4-Lite**	**AXI4-Stream**
Addressing	[**TODO**]	[**TODO**]	[**TODO**]
Channels	[**TODO**]	[**TODO**]	[**TODO**]
Bursts	[**TODO**]	[**TODO**]	[**TODO**]
Typical use	[**TODO**]	[**TODO**]	[**TODO**]

1.3 Terminology: transfer, packet, frame, stream

[**TODO**] Define each level of the hierarchy and how they map to signals.

1.4 Roadmap of the document

[**TODO**] One paragraph per part.

Chapter 2

The AXI4-Stream Protocol

2.1 Signal set

Table 2.1: AXI4-Stream signal summary.

Signal	Source	Width	Purpose
`ACLK`	Clock	1	[TODO]
`ARESETn`	Reset	1	[TODO]
`TVALID`	Master	1	[TODO]
`TREADY`	Slave	1	[TODO]
`TDATA`	Master	$8n$	[TODO]
`TSTRB`	Master	n	[TODO]
`TKEEP`	Master	n	[TODO]
`TLAST`	Master	1	[TODO]
`TID`	Master	user	[TODO]
`TDEST`	Master	user	[TODO]
`TUSER`	Master	user	[TODO]

2.1.1 Required vs. optional signals

[TODO] Minimal legal interface; default values for omitted signals.

2.1.2 Byte qualifiers: data, position and null bytes

[TODO] Table of `TKEEP`/`TSTRB` encodings and their meaning.

2.2 The VALID/READY handshake

Protocol rule

[TODO] State the three canonical handshake rules: (1) a transfer occurs when `TVALID` and `TREADY` are both high on a rising clock edge; (2) once asserted, `TVALID` must remain asserted until the transfer completes; (3) `TREADY` may be asserted or deasserted freely.

2.2.1 Case 1: master first

[TODO] Waveform + description.

2.2.2 Case 2: slave first

[**TODO**] Waveform + description.

2.2.3 Case 3: simultaneous assertion

[**TODO**] Waveform + description.



Figure 2.1: Basic AXI4-Stream handshake with back-to-back transfers.

Common pitfall

[**TODO**] `TVALID` must not depend combinationally on `TREADY`. Explain why, and show the deadlock it creates.

2.3 Packet boundaries with `TLAST`

[**TODO**] Packet framing, aligned vs. unaligned, zero-length packets.

2.4 Routing with `TID` and `TDEST`

[**TODO**] Stream identification, ordering guarantees, interconnect implications.

2.5 Sideband information with `TUSER`

[**TODO**] Per-transfer vs. per-byte `TUSER`; sets up the video profile in chapter 7.

2.6 Reset behaviour

[**TODO**] `ARESETn` requirements; `TVALID` must be low at reset release.

2.7 Interface compliance checklist

[**TODO**] Bullet list a reviewer can run down.

Chapter 3

Timing, Throughput and Backpressure

3.1 Throughput model

[**TODO**] Effective bandwidth = $f_{\text{clk}} \times W_{\text{bytes}} \times \eta$; define the efficiency term η.

3.2 Bubbles and stalls

[**TODO**] Sources: source starvation, sink backpressure, pipeline hazards.

3.3 Registering the interface

3.3.1 Naive output register (breaks throughput)

3.3.2 The skid buffer / register slice

[**TODO**] Full-throughput, fully-registered pass-through. Include RTL.

```
// TODO: implement
module axis_skid_buffer #(
    parameter integer DATA_W = 32
)(
    input  wire                  aclk,
    input  wire                  aresetn,
    // slave side
    input  wire [DATA_W-1:0]     s_axis_tdata,
    input  wire                  s_axis_tvalid,
    output wire                  s_axis_tready,
    input  wire                  s_axis_tlast,
    // master side
    output wire [DATA_W-1:0]     m_axis_tdata,
    output wire                  m_axis_tvalid,
    input  wire                  m_axis_tready,
    output wire                  m_axis_tlast
);
endmodule
```

Listing 3.1: Skid buffer skeleton.

3.4 FIFOs and elastic buffering

[**TODO**] Sizing rules, almost-full thresholds, clock-domain crossing.

3.5 Deadlock and starvation

[**TODO**] Cyclic dependencies in multi-stream designs.

Chapter 4

Writing AXI4-Stream Masters and Slaves

4.1 A minimal master

[**TODO**] Reference `m_axis.v`.

```
1 // TODO: fill in from m_axis.v
```

Listing 4.1: AXI4-Stream master port declaration.

4.2 A minimal slave

```
1 // TODO: fill in from s_axis.v
```

Listing 4.2: AXI4-Stream slave port declaration.

4.3 Pass-through and processing elements

4.3.1 Zero-latency (combinational) processing

4.3.2 Pipelined processing with backpressure

[**TODO**] The `ready = !valid_reg || downstream_ready` idiom.

4.4 Rate-changing elements

[**TODO**] Upsizers, downsizers, decimators, and how `TLAST`/`TKEEP` must be regenerated.

4.5 Parameterization and reuse conventions

[**TODO**] Coding style used throughout the repository.

Chapter 5

Stream Infrastructure Blocks

5.1 Register slice

5.2 Data FIFO

5.3 Width converter

5.4 Broadcaster (1-to-N)

5.5 Combiner and switch (N-to-1, N-to-M)

5.6 Subset converter

[**TODO**] For each: purpose, parameters, latency, and a small usage example.

Part II

AXI4-Stream Video

Chapter 6

Digital Video Fundamentals

6.1 Raster scan, blanking and active video

[**TODO**] Diagram of a frame with horizontal/vertical blanking regions.

6.2 Video timing parameters

Table 6.1: Common video timing modes.

Mode	H active	V active	Pixel clock	Frame rate	Note
640x480p60	640	480	[TODO]	[TODO]	[TODO]
1280x720p60	1280	720	[TODO]	[TODO]	[TODO]
1920x1080p60	1920	1080	[TODO]	[TODO]	[TODO]
3840x2160p60	3840	2160	[TODO]	[TODO]	[TODO]

6.3 Colour spaces and pixel formats

[**TODO**] RGB, YCbCr 4:4:4 / 4:2:2 / 4:2:0, bit depths.

6.4 Bandwidth budgeting

[**TODO**] Worked example: pixels/clock needed for a given mode at a given f_{clk}.

Chapter 7

The AXI4-Stream Video Protocol

7.1 Mapping video onto AXI4-Stream

[**TODO**] Only active video is transported; blanking is not.

Table 7.1: Video-specific interpretation of the AXI4-Stream signals.

Signal	Video meaning	Comment
`TDATA`	Pixel data	[**TODO**]
`TUSER[0]`	Start of frame (SOF)	[**TODO**]
`TLAST`	End of line (EOL)	[**TODO**]
`TVALID`	Pixel valid	[**TODO**]
`TREADY`	Downstream ready	[**TODO**]
`TKEEP`	Usually tied high	[**TODO**]

7.2 Start of frame and end of line

Protocol rule

[**TODO**] SOF is asserted on the first pixel of the first line; EOL on the last pixel of every line, including the last line of the frame.

[**TODO**] Figure: SOF/EOL over a 4x3 frame.

Figure 7.1: Placement of SOF and EOL over a small frame.

7.3 Pixel packing in `TDATA`

7.3.1 One pixel per clock

7.3.2 Multiple pixels per clock

7.3.3 Component ordering and padding

[**TODO**] Bit-level layout tables for RGB888, YUV422, and grey8.

7.4 Frame synchronisation and error recovery

[**TODO**] What a downstream block must do when SOF arrives early/late; line-length mismatch handling.

7.5 Differences from generic AXI4-Stream

[**TODO**] Summary table: what the video profile constrains or repurposes.

Common pitfall

[**TODO**] Forgetting that `TUSER` SOF is only valid on a qualified transfer (`TVALID` & `TREADY`).

Chapter 8

Video Timing and the Boundary Blocks

8.1 Video Timing Controller (VTC)

[**TODO**] Generating and detecting timing.

8.2 From parallel video to AXI4-Stream

[**TODO**] Video-In-to-AXI4-Stream: hsync/vsync/de → SOF/EOL.

8.3 From AXI4-Stream to parallel video

[**TODO**] AXI4-Stream-to-Video-Out: locking, the "no backpressure" problem, underflow/overflow behaviour.

8.4 Clock domains in a video pipeline

[**TODO**] Pixel clock vs. processing clock; where to place the async FIFO.

Chapter 9

Building a Video Processing Pipeline

9.1 Anatomy of a pipeline

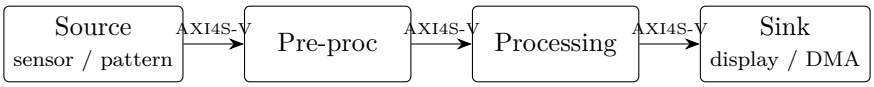


Figure 9.1: Skeleton of an AXI4-Stream video pipeline.

9.2 Point operations

[**TODO**] Colour conversion, gamma, thresholding — stateless, latency 1.

9.3 Line-based operations

[**TODO**] Line buffers, and how they interact with EOL.

9.4 Neighbourhood operations

[**TODO**] 3x3 window generator, border handling.

9.5 Frame buffers and DMA

[**TODO**] VDMA / Frame Buffer Read & Write, memory bandwidth, tearing.

9.6 Worked example: grayscale converter

[**TODO**] Full RTL walkthrough, from ports to simulation results.

Chapter 10

Verification and Debug

10.1 Testbench architecture

[**TODO**] Stimulus generator, monitor, scoreboard.

10.2 Randomised backpressure and valid throttling

[**TODO**] Why constant-ready testbenches hide bugs.

```
// TODO: implement
```

Listing 10.1: Randomised `TREADY` driver.

10.3 Protocol checkers and assertions

[**TODO**] SVA properties for the handshake and for SOF/EOL placement.

10.4 Image-level checking

[**TODO**] Dumping frames to PPM/PGM and comparing against a reference model.

10.5 On-hardware debug

[**TODO**] ILA trigger recipes; counting SOF/EOL; detecting stuck `TREADY`.

10.6 Symptom-to-cause table

Table 10.1: Debugging AXI4-Stream video pipelines.

Symptom	Likely cause
Image shifted horizontally	[**TODO**]
Image rolls vertically	[**TODO**]
Pipeline hangs after one frame	[**TODO**]
Intermittent tearing	[**TODO**]
Black output, `TVALID` low	[**TODO**]

Appendix A

Signal Quick Reference

[**TODO**] One-page card: all signals, widths, sources, defaults.

Appendix B

Design Review Checklist

[**TODO**] Checklist for an AXI4-Stream video block before integration.

Appendix C

Glossary

`SOF` Start of frame. **[TODO]**

`EOL` End of line. **[TODO]**

`Beat`
: **[TODO]**

`Backpressure`
: **[TODO]**

`Skid buffer`
: **[TODO]**

Bibliography

[1] ARM, *AMBA 4 AXI4-Stream Protocol Specification*, IHI 0051. **[TODO]** Confirm the revision used.

[2] AMD/Xilinx, *AXI4-Stream Video IP and System Design Guide*, UG934.

[3] AMD/Xilinx, *Vivado AXI Reference Guide*, UG1037.

[TODO] Add further references.